

Turno

Banco de dados

Diagrama de relacionamentos e arquitetura do banco que o motor de tarefas distribuídas deve seguir

Feito por Erik L. Pimenta

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

1 **Visão geral** 4

Princípios 4

Banco suportado 4

Convenções 4

As tabelas 5

2 **Diagrama de relacionamentos** 6

3 **Relacionamentos e outbox** 7

Chaves estrangeiras 7

Outbox transacional 7

4 **Tabelas** 9

jobs 9

job_attempts 10

job_state_changes 11

recurring_jobs 11

workers 12

dead_letters 13

outbox_state 13

outbox_message 14

inbox_state 14

5 **Regras de consistência** 16

Ciclo de estados 16

Início de uma execução 16

Renovação do lease e cerca 17

Recuperação de workers perdidos 17

Retenção 17

6 **Como criar e parear o banco** 18

Com Docker 18

Em um servidor existente 18

Gerando o script SQL para um DBA 18

Evoluindo o esquema 18

Capítulo 1

Visão geral

Este documento descreve o modelo de dados que o Turno usa para coordenar tarefas entre vários workers: as tabelas, os campos, as chaves, os índices e as regras que garantem que cada tarefa rode uma única vez por tentativa. Qualquer banco que hospede o Turno precisa seguir este modelo.

Princípios

- **O banco é a fonte da verdade.** O RabbitMQ (ou o Azure Service Bus) só transporta o aviso de que existe trabalho. Estado, tentativas, lease e histórico ficam no PostgreSQL.
- **Toda mudança de estado é um `UPDATE` condicional.** Os workers nunca leem, decidem e gravam em passos separados: a condição vai no próprio `WHERE` e o número de linhas afetadas diz se a operação valeu.
- **Mensagens saem pela outbox.** O envio para o broker é gravado na mesma transação da tarefa, nas tabelas do MassTransit, e entregue depois do commit.
- **Histórico preservado.** Tentativas e mudanças de estado guardam o identificador do worker como texto, sem chave estrangeira, para continuarem legíveis depois que workers antigos são removidos.

Banco suportado

Item	Valor
Banco	PostgreSQL 14 ou mais recente (testado com 16 e 17)
Schema	`turno`, criado pela primeira migração
Acesso	Entity Framework Core 10 com o provedor Npgsql
Migrações	Pasta `src/Turno.Infrastructure/Persistence/Migrations`; histórico em `turno.__ef_migrations_history`
Aplicação	Automática na subida da API, protegida por `pg_advisory_lock`; os workers esperam o esquema ficar pronto

Convenções

Convenção	Como é aplicada
Nomes	Tabelas, colunas, chaves e índices em `snake_case`, com prefixos `pk_`, `fk_`, `ix_`, `ak_` e `ck_`.
Datas	Sempre `timestamptz` gravado em UTC. A conversão para o fuso do usuário acontece no painel.
Estados	Enumerações gravadas como texto (`varchar(20)`), legíveis em consultas diretas.

Convenção	Como é aplicada
Identificadores	UUID v7, ordenado por tempo, nas tabelas que recebem muitas inserções; `bigint identity` nas tabelas de histórico.
Dados livres	Payload e resultado em `jsonb`; filas e tipos atendidos pelos workers em `text[]`.
Concorrência	Atualizações condicionais por estado e token de despacho nas tarefas; coluna de sistema `xmin` como controle otimista nos agendamentos.

As tabelas

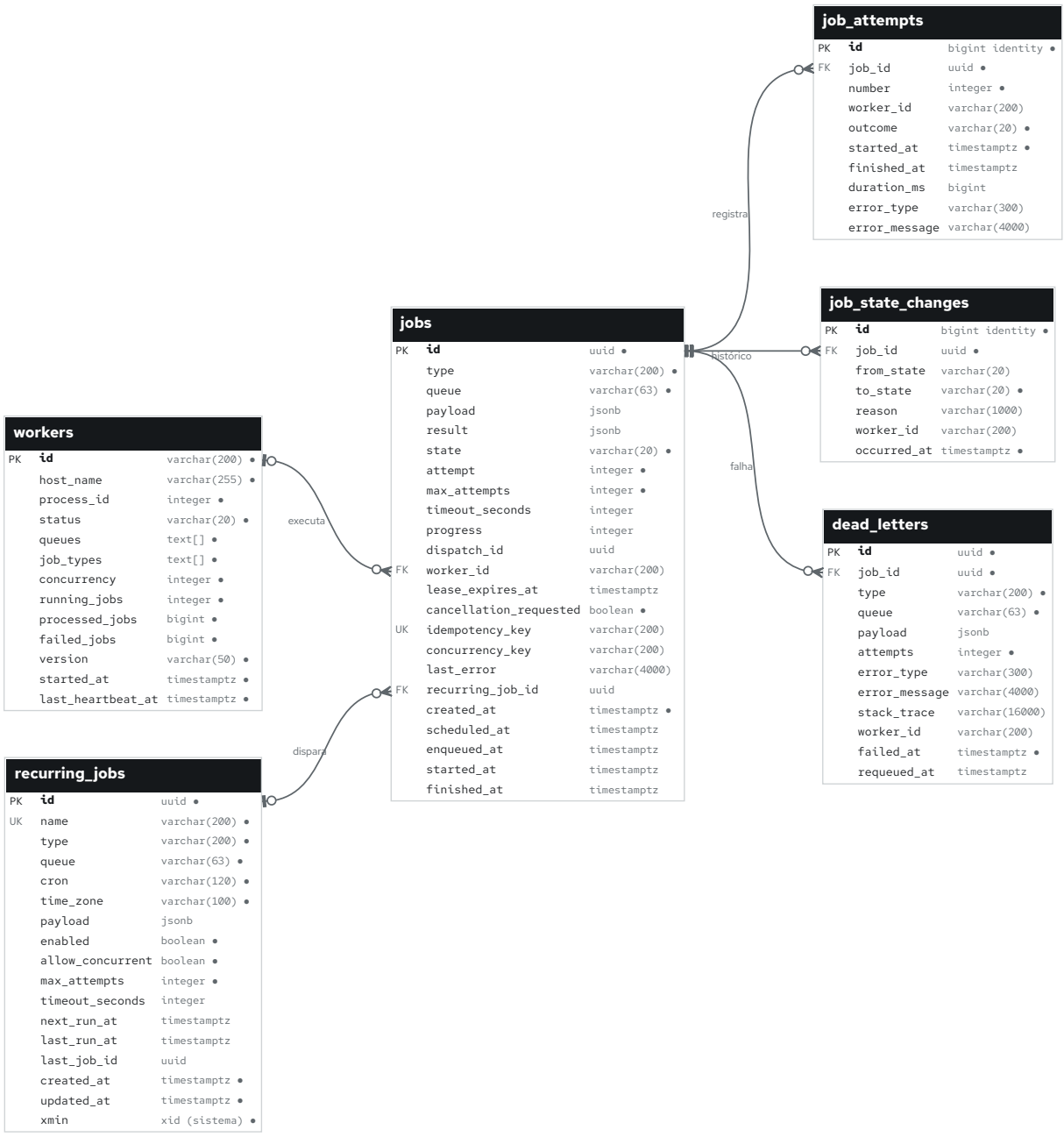
Grupo	Tabela	O que guarda
Tarefas	`jobs`	Cada tarefa, com estado, tentativas, token de despacho e lease.
	`job_attempts`	Cada execução iniciada e como terminou.
	`job_state_changes`	Linha do tempo de mudanças de estado.
	`dead_letters`	Fila de erros.
Agendamento	`recurring_jobs`	Agendamentos cron.
Cluster	`workers`	Workers registrados e sinal de vida.
Mensageria	`outbox_message`, `outbox_state`, `inbox_state`	Outbox transacional do MassTransit.

Capítulo 2

Diagrama de relacionamentos

Tabelas do domínio do Turno com todos os campos. A tarefa (`jobs`) é o centro: tudo o que acontece com ela fica registrado nas tabelas à direita.

PK chave primária FK chave estrangeira UK valor único ● obrigatório (NOT NULL) || exatamente um |o zero ou um o< zero ou muitos



Capítulo 3

Relacionamentos e outbox

Como as tabelas se ligam, o que acontece em cada exclusão e onde ficam as tabelas de mensageria.

Chaves estrangeiras

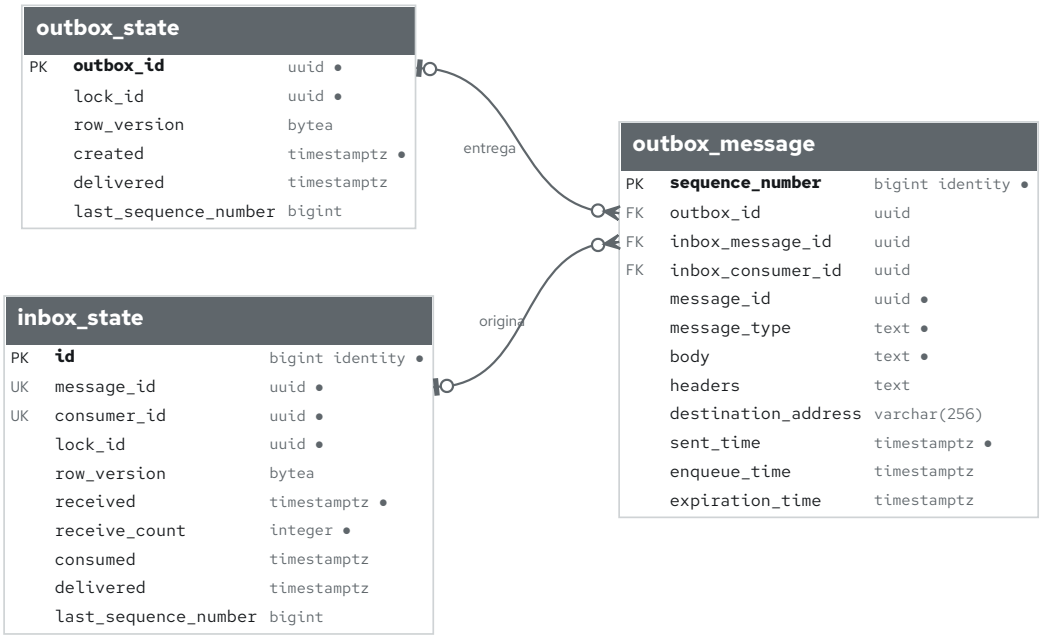
Coluna	Referência	Cardinalidade	Ao excluir o pai	Significado
`jobs.recurring_job_id`	`recurring_jobs.id`	0..1 : N	SET NULL	dispara
`jobs.worker_id`	`workers.id`	0..1 : N	SET NULL	executa
`job_attempts.job_id`	`jobs.id`	1 : N	CASCADE	registra
`job_state_changes.job_id`	`jobs.id`	1 : N	CASCADE	histórico
`dead_letters.job_id`	`jobs.id`	1 : N	CASCADE	falha

Três referências são propositalmente lógicas, sem chave estrangeira:

- `job_attempts.worker_id`, `job_state_changes.worker_id` e `dead_letters.worker_id` guardam o worker como texto para o histórico sobreviver à limpeza de workers offline.
- `recurring_jobs.last_job_id` aponta para a última tarefa criada sem formar um ciclo de chaves com `jobs.recurring_job_id`.

Outbox transacional

As tabelas abaixo são criadas pelo MassTransit dentro do mesmo schema. Quando a API cria uma tarefa, a linha em `jobs`, a linha em `job_state_changes` e a mensagem `ExecuteJob` em `outbox_message` entram na mesma transação. O serviço de entrega do MassTransit publica a mensagem depois do commit e marca o lote em `outbox_state`.


outbox_state
PK outbox_id uuid •
lock_id uuid •
row_version bytea
created timestamptz •
delivered timestamptz
last_sequence_number bigint
entrega
outbox_message
PK sequence_number bigint identity •
FK outbox_id uuid
FK inbox_message_id uuid
FK inbox_consumer_id uuid
message_id uuid •
message_type text •
body text •
headers text
destination_address varchar(256)
sent_time timestamptz •
enqueue_time timestamptz
expiration_time timestamptz
origina
inbox_state
PK id bigint identity •
UK message_id uuid •
UK consumer_id uuid •
lock_id uuid •
row_version bytea
received timestamptz •
receive_count integer •
consumed timestamptz
delivered timestamptz
last_sequence_number bigint

Capítulo 4

Tabelas

Campos, tipos, chaves, índices e restrições de cada tabela. Os tipos são os gerados pela migração inicial para o PostgreSQL.

jobs

Cada tarefa enfileirada ou agendada. É a fonte da verdade do estado; o broker só transporta o aviso de que há trabalho.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	`uuid`	não	PK	Identificador ordenado por tempo (UUID v7).
`type`	`varchar(200)`	não		Tipo da tarefa, ligado a um handler, por exemplo report.generate.
`queue`	`varchar(63)`	não		Fila de destino no broker (default, critical, reports...).
`payload`	`jsonb`	sim		Dados de entrada do handler (até 256 KB).
`result`	`jsonb`	sim		Resultado gravado pelo handler em caso de sucesso (até 32 KB).
`state`	`varchar(20)`	não		Scheduled, Enqueued, Processing, Succeeded, Failed ou Cancelled.
`attempt`	`integer`	não		Tentativas já contadas. Adiamentos e interrupções não contam.
`max_attempts`	`integer`	não		Limite de tentativas antes da fila de erros (1 a 100).
`timeout_seconds`	`integer`	sim		Tempo limite por tentativa; nulo usa o padrão do tipo.
`progress`	`integer`	sim		Progresso informado pelo handler, de 0 a 100.
`dispatch_id`	`uuid`	sim		Token de despacho. Muda a cada envio para a fila; só a mensagem com o token atual inicia a tarefa.
`worker_id`	`varchar(200)`	sim	FK	Worker que executa ou executou por último.
`lease_expires_at`	`timestamptz`	sim		Fim do lease da execução atual. Vencido, a tarefa é recuperada.
`cancellation_requested`	`boolean`	não		Pedido de cancelamento para uma execução em andamento.
`idempotency_key`	`varchar(200)`	sim	UK	Chave enviada pelo cliente para evitar duplicatas.

Coluna	Tipo	Nulo	Chave	Descrição
`concurrency_key`	`varchar(200)`	sim		Tarefas com a mesma chave nunca rodam ao mesmo tempo.
`last_error`	`varchar(4000)`	sim		Último erro registrado.
`recurring_job_id`	`uuid`	sim	FK	Agendamento recorrente que disparou a tarefa.
`created_at`	`timestamptz`	não		Criação.
`scheduled_at`	`timestamptz`	sim		Momento em que pode voltar para a fila.
`enqueued_at`	`timestamptz`	sim		Último envio para a fila.
`started_at`	`timestamptz`	sim		Início da última tentativa.
`finished_at`	`timestamptz`	sim		Término (sucesso, falha definitiva ou cancelamento).

Índices: `pk_jobs` (id); `ix_jobs_idempotency_key` (idempotency_key) único; `ix_jobs_state_scheduled_at` (state, scheduled_at): agendador; `ix_jobs_state_lease_expires_at` (state, lease_expires_at): recuperação; `ix_jobs_queue_state` (queue, state): filas no painel; `ix_jobs_type_created_at` (type, created_at); `ix_jobs_created_at` (created_at); `ix_jobs_state_finished_at` (state, finished_at): retenção; `ix_jobs_recurring_job_id` (recurring_job_id); `ix_jobs_worker_id` (worker_id).

Restrições: `ck_jobs_attempt: attempt >= 0`; `ck_jobs_max_attempts: max_attempts BETWEEN 1 AND 100`; `ck_jobs_progress: progress IS NULL OR progress BETWEEN 0 AND 100`; `ck_jobs_timeout: timeout_seconds IS NULL OR timeout_seconds > 0`.

job_attempts

Uma linha por execução iniciada. Guarda quem executou, quanto tempo levou e como terminou.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	`bigint identity`	não	PK	Sequencial.
`job_id`	`uuid`	não	FK	Tarefa executada.
`number`	`integer`	não		Número da tentativa no momento do início.
`worker_id`	`varchar(200)`	sim		Worker que executou (referência lógica, preservada após a limpeza de workers).
`outcome`	`varchar(20)`	não		Running, Succeeded, Failed, Deferred, Interrupted, Lost ou Cancelled.
`started_at`	`timestamptz`	não		Início.
`finished_at`	`timestamptz`	sim		Fim.
`duration_ms`	`bigint`	sim		Duração em milissegundos.

Coluna	Tipo	Nulo	Chave	Descrição
`error_type`	`varchar(300)`	sim		Tipo completo da exceção.
`error_message`	`varchar(4000)`	sim		Mensagem do erro ou motivo do adiamento.

Índices: `pk_job_attempts` (id); `ix_job_attempts_job_id_started_at` (job_id, started_at); `ix_job_attempts_finished_at_outcome` (finished_at, outcome): vazão por minuto.

Restrições: `ck_job_attempts_number: number >= 1`.

job_state_changes

Linha do tempo de cada tarefa: toda mudança de estado com o motivo legível.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	`bigint identity`	não	PK	Sequencial; define a ordem dos eventos.
`job_id`	`uuid`	não	FK	Tarefa.
`from_state`	`varchar(20)`	sim		Estado anterior; nulo na criação.
`to_state`	`varchar(20)`	não		Novo estado.
`reason`	`varchar(1000)`	sim		Motivo, por exemplo "Falhou na tentativa 2 de 5. Nova tentativa em 30 s."
`worker_id`	`varchar(200)`	sim		Worker envolvido (referência lógica).
`occurred_at`	`timestamptz`	não		Momento da mudança.

Índices: `pk_job_state_changes` (id); `ix_job_state_changes_job_id_occurred_at` (job_id, occurred_at).

recurring_jobs

Agendamentos por expressão cron. O líder do agendador cria uma tarefa a cada ocorrência.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	`uuid`	não	PK	Identificador.
`name`	`varchar(200)`	não	UK	Nome único, usado para criar ou atualizar pelo appsettings e pela API.
`type`	`varchar(200)`	não		Tipo da tarefa criada.
`queue`	`varchar(63)`	não		Fila das tarefas criadas.
`cron`	`varchar(120)`	não		Expressão cron com 5 campos, ou 6 com segundos.
`time_zone`	`varchar(100)`	não		Fuso IANA, por exemplo America/Sao_Paulo.

Coluna	Tipo	Nulo	Chave	Descrição
`payload`	`jsonb`	sim		Payload de cada execução.
`enabled`	`boolean`	não		Ativo ou pausado.
`allow_concurrent`	`boolean`	não		Se falso, pula a ocorrência enquanto a anterior não terminar.
`max_attempts`	`integer`	não		Tentativas de cada tarefa criada.
`timeout_seconds`	`integer`	sim		Tempo limite de cada tarefa criada.
`next_run_at`	`timestamptz`	sim		Próxima ocorrência em UTC.
`last_run_at`	`timestamptz`	sim		Último disparo.
`last_job_id`	`uuid`	sim		Última tarefa criada (referência lógica, evita FK circular).
`created_at`	`timestamptz`	não		Criação.
`updated_at`	`timestamptz`	não		Última alteração.
`xmin`	`xid (sistema)`	não		Coluna de sistema do PostgreSQL usada como controle otimista de concorrência.

Índices: `pk_recurring_jobs` (id); `ix_recurring_jobs_name` (name) único; `ix_recurring_jobs_enabled_next_run_at` (enabled, next_run_at).

Restrições: `ck_recurring_jobs_max_attempts: max_attempts BETWEEN 1 AND 100`; `ck_recurring_jobs_timeout: timeout_seconds IS NULL OR timeout_seconds > 0`.

workers

Instâncias de Turno.Worker registradas. Atualizadas pelo heartbeat a cada 5 s.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	`varchar(200)`	não	PK	Nome do worker + sufixo aleatório, novo a cada processo.
`host_name`	`varchar(255)`	não		Nome da máquina ou do contêiner.
`process_id`	`integer`	não		PID do processo.
`status`	`varchar(20)`	não		Online, Stopping ou Offline.
`queues`	`text[]`	não		Filas consumidas.
`job_types`	`text[]`	não		Tipos de tarefa com handler registrado.
`concurrency`	`integer`	não		Total de vagas de execução.
`running_jobs`	`integer`	não		Execuções em andamento no último sinal.

Coluna	Tipo	Nulo	Chave	Descrição
`processed_jobs`	`bigint`	não		Execuções concluídas desde o início do processo.
`failed_jobs`	`bigint`	não		Execuções com falha desde o início do processo.
`version`	`varchar(50)`	não		Versão do Turno.
`started_at`	`timestamptz`	não		Início do processo.
`last_heartbeat_at`	`timestamptz`	não		Último sinal de vida.

Índices: `pk_workers` (id); `ix_workers_status_last_heartbeat_at` (status, last_heartbeat_at).

Restrições: `ck_workers_concurrency: concurrency >= 0 AND running_jobs >= 0`.

dead_letters

Fila de erros. Uma linha cada vez que uma tarefa esgota as tentativas ou recebe erro permanente.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	`uuid`	não	PK	Identificador.
`job_id`	`uuid`	não	FK	Tarefa original, que fica em Failed.
`type`	`varchar(200)`	não		Tipo da tarefa.
`queue`	`varchar(63)`	não		Fila.
`payload`	`jsonb`	sim		Cópia do payload no momento da falha.
`attempts`	`integer`	não		Tentativas realizadas.
`error_type`	`varchar(300)`	sim		Tipo da exceção ou WorkerLost.
`error_message`	`varchar(4000)`	sim		Mensagem.
`stack_trace`	`varchar(16000)`	sim		Stack trace completo.
`worker_id`	`varchar(200)`	sim		Worker da última tentativa.
`failed_at`	`timestamptz`	não		Momento da falha definitiva.
`requeued_at`	`timestamptz`	sim		Quando foi reenfileirada; nulo enquanto pendente.

Índices: `pk_dead_letters` (id); `ix_dead_letters_failed_at` (failed_at); `ix_dead_letters_job_id_requeued_at` (job_id, requeued_at).

outbox_state

Controle de cada lote de mensagens gravado pelo outbox (uma linha por transação que enviou mensagens).

Coluna	Tipo	Nulo	Chave	Descrição
`outbox_id`	`uuid`	não	PK	Identificador do lote.
`lock_id`	`uuid`	não		Lock usado pelo serviço de entrega.
`row_version`	`bytea`	sim		Controle de concorrência.
`created`	`timestamptz`	não		Criação.
`delivered`	`timestamptz`	sim		Entrega concluída.
`last_sequence_number`	`bigint`	sim		Última mensagem entregue.

Índices: `pk_outbox_state` (outbox_id); `ix_outbox_state_created` (created).

outbox_message

Mensagens aguardando envio ao broker, gravadas na mesma transação das tarefas.

Coluna	Tipo	Nulo	Chave	Descrição
`sequence_number`	`bigint identity`	não	PK	Ordem de envio.
`outbox_id`	`uuid`	sim	FK	Lote do outbox.
`inbox_message_id`	`uuid`	sim	FK	Mensagem de entrada (outbox de consumidor).
`inbox_consumer_id`	`uuid`	sim	FK	Consumidor de entrada.
`message_id`	`uuid`	não		Identificador da mensagem.
`message_type`	`text`	não		Tipo da mensagem (URN).
`body`	`text`	não		Conteúdo serializado.
`headers`	`text`	sim		Cabeçalhos.
`destination_address`	`varchar(256)`	sim		Fila ou exchange de destino.
`sent_time`	`timestamptz`	não		Momento do envio pela aplicação.
`enqueue_time`	`timestamptz`	sim		Entrega agendada.
`expiration_time`	`timestamptz`	sim		Expiração.

Índices: `pk_outbox_message` (sequence_number); `ix_outbox_message_outbox_id_sequence_number` (outbox_id, sequence_number) único; `ix_outbox_message_enqueue_time` ; `ix_outbox_message_expiration_time` .

inbox_state

Deduplicação de mensagens recebidas, usada pelo outbox de consumidor do MassTransit.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	`bigint identity`	não	PK	Sequencial.
`message_id`	`uuid`	não	UK	Mensagem recebida.
`consumer_id`	`uuid`	não	UK	Consumidor.
`lock_id`	`uuid`	não		Lock de processamento.
`row_version`	`bytea`	sim		Controle de concorrência.
`received`	`timestamptz`	não		Recebimento.
`receive_count`	`integer`	não		Vezes recebida.
`consumed`	`timestamptz`	sim		Consumo concluído.
`delivered`	`timestamptz`	sim		Saídas entregues.
`last_sequence_number`	`bigint`	sim		Última saída.

Índices: `pk_inbox_state` (id); `ak_inbox_state_message_id_consumer_id` (message_id, consumer_id) único; `ix_inbox_state_delivered` (delivered).

Capítulo 5

Regras de consistência

As consultas que mantêm o cluster correto. Todas usam a condição no próprio UPDATE, então duas instâncias nunca obtêm o mesmo resultado.

Ciclo de estados

De	Para	Quando
(novo)	Scheduled	Criada com data futura ou atraso.
(novo)	Enqueued	Criada para agora; recebe um `dispatch_id` e a mensagem vai para a outbox.
Scheduled	Enqueued	O líder do agendador encontra `scheduled_at <= agora`.
Enqueued	Processing	Um worker obteve o lock Redlock e o `UPDATE` com o mesmo `dispatch_id` afetou 1 linha.
Enqueued	Scheduled	Lock da tarefa ou da chave de concorrência em uso; volta em segundos.
Processing	Succeeded	Handler terminou e o worker ainda é dono do lease.
Processing	Scheduled	Falhou com tentativas restantes (backoff), disjuntor aberto ou worker encerrado.
Processing	Enqueued	Lease vencido: o worker parou de responder. Nova mensagem com novo `dispatch_id`.
Processing	Failed	Tentativas esgotadas ou erro permanente. Uma linha vai para `dead_letters`.
Processing	Cancelled	Cancelamento pedido e atendido pelo handler.
Scheduled, Enqueued	Cancelled	Cancelamento antes de iniciar.
Failed, Cancelled	Enqueued	Reenfileirada pelo painel ou pela API; tentativas voltam a zero.

Início de uma execução

```
UPDATE turno.jobs
SET state = 'Processing',
    attempt = attempt + 1,
    worker_id = @worker,
    started_at = now(),
    lease_expires_at = now() + interval '60 seconds',
    finished_at = NULL,
    progress = NULL
WHERE id = @job
  AND dispatch_id = @dispatch
  AND state = 'Enqueued'
  AND NOT cancellation_requested;
```

Se nenhuma linha for afetada, a mensagem era duplicada ou antiga e é descartada. Na mesma transação entram a linha em `job_attempts` com `outcome = 'Running'` e a mudança em `job_state_changes`.

Renovação do lease e cerca

```
UPDATE turno.jobs
SET lease_expires_at = now() + interval '60 seconds'
WHERE id = @job
  AND dispatch_id = @dispatch
  AND worker_id = @worker
  AND state = 'Processing';
```

As mesmas quatro condições protegem a gravação do resultado. Se o lease foi assumido por outro worker, o `dispatch_id` mudou e o resultado do worker antigo é descartado: esse é o token de cerca que complementa o Redlock.

Recuperação de workers perdidos

```
SELECT id, attempt, max_attempts, dispatch_id, lease_expires_at
FROM turno.jobs
WHERE state = 'Processing'
  AND lease_expires_at < now() - interval '15 seconds'
ORDER BY lease_expires_at
LIMIT 100;
```

Para cada linha, o líder de recuperação repete o `UPDATE` condicionando também `dispatch_id` e `lease_expires_at` lidos, gera um novo token e devolve a tarefa à fila, ou a envia para a fila de erros se as tentativas acabaram. A tentativa perdida fica com `outcome = 'Lost'`.

Retenção

Dado	Removido após	Configuração
Tarefas com sucesso	7 dias	`Turno:Retention:SucceededJobs`
Tarefas canceladas	7 dias	`Turno:Retention:CancelledJobs`
Tarefas com falha (e suas linhas em `dead_letters`)	30 dias	`Turno:Retention:FailedJobs`
Workers offline	1 dia	`Turno:Retention:OfflineWorkers`

Tentativas e histórico são removidos em cascata junto com a tarefa.

Capítulo 6

Como criar e parear o banco

Três caminhos, do mais simples ao mais controlado.

Com Docker

O `docker-compose.yml` sobe o PostgreSQL 17 com usuário, senha e banco vindos do `.env`. A API aplica as migrações na primeira subida.

```
docker compose up -d postgres
```

Em um servidor existente

```
CREATE USER turno WITH PASSWORD '<defina uma senha forte>';
CREATE DATABASE turno OWNER turno;
```

Depois aponte a aplicação para o banco pela variável de ambiente ou pelos user-secrets:

```
ConnectionStrings__Turno="Host=<servidor>;Port=5432;Database=turno;Username=turno;Password=
<senha>"
```

Gerando o script SQL para um DBA

Quando a aplicação não pode alterar o esquema em produção, desligue a migração automática com `Turno__Database__ApplyMigrations=false` e gere um script idempotente:

```
dotnet tool install --global dotnet-ef
dotnet ef migrations script --idempotent \
  --project src/Turno.Infrastructure \
  --startup-project src/Turno.Api \
  --output turno.sql
```

Evoluindo o esquema

```
dotnet ef migrations add NomeDaMudanca \
  --project src/Turno.Infrastructure \
  --startup-project src/Turno.Api \
  --output-dir Persistence/Migrations
```

O teste `Migrations_match_the_model` falha se o modelo mudar sem uma migração correspondente, então a integração contínua avisa antes de o problema chegar a produção.